Harish-Chandra Research Institute
Allahabad, India

Grover's algorithm for unstructured search

A computational simulation of the quantum mechanical algorithm for database search

Anirudha Bose
Department of Computer Science & Engineering
IIIT Bhubaneswar
Email: ani07nov@gmail.com

Mentor
Dr. Prabhu Ramappa
INSPIRE Faculty
Quantum Information and Computation (QIC) group
HRI, Allahabad
Email: prabhurama@hri.res.in

Last updated on: November 16, 2018
Cite as: IIITBH-ACM/TR/2014/001
Permalink: http://iiitbh-acm.github.io/IIITBH-ACM_TR_2014_001.pdf

This work was supported by Harish-Chandra Research Institute (HRI), Allahabad under the Visiting Student Programme (VSP) during the period September 2014 to October 2014. The author would like to thank Dr. R. Prabhu for his guidance and the entire QIC group at HRI.

All code samples and text in this document unless otherwise stated are released under the MIT License.

Department of Computer Science & Engineering
International Institute of Information Technology
Bhubaneswar, India

1 Computational Hardness

A ”decision problem” can be modelled as:

$$f : \{0,1\}^n \mapsto \{0,1\}$$

A resource independent way of quantifying the hardness of a problem is to analyse how the size of a circuit scales with the input size n. The above problem scales exponentially (2^n) with input size n.

We say that a decision problem is in P if the algorithm (or circuit family) can be described as a polynomial function of input size n. As a matter of fact, most of the functions described by the above mapping are not in P. For such functions, the only way to determine $f(x)$ is to search a table for the solution out of exponentially many possible entries. This table can be considered as the digital equivalent of a haystack, and the solution to the problem is a needle which we are interested in finding. Hence finding a solution to an NP-complete [a] problem can be seen as a search problem represented by the following Boolean satisfiability [b].

$$\boxed{f(x_1, x_2, x_3, ..., x_n) = (x_1 \vee x_2' \vee x_3) \wedge (x_2 \vee x_5' \vee x_6) \wedge ...}$$

The satisfiability problem is to find a configuration $x_1, x_2, x_3, ..., x_n$ such that $f(x_1, x_2, x_3, ..., x_n) = 1$. It is clear that the problem requires us to look up a haystack of 2^n entries and find a needle. In fact there are thousands of NP-complete problems which are computationally equivalent to satisfiability. The problems in P are the ones which have enough structure in them to be computationally easy to solve. An NP-complete problem is hard to solve because it is equivalent to searching an unstructured database for an element.

---

[a] NP stands for ”non-deterministic polynomial time”. If a decision problem is in NP, it may not be possible to determine the solution in polynomial time, but easy to verify the correctness of the solution once we find it. For example, the prime factorization problem is in NP. A problem is NP-Complete if any problem in NP is polynomially reducible to it. Example, CIRCUIT-SAT. It can also be shown that CIRCUIT-SAT problem is reducible to Hamiltonian Path problem making the latter NP-Complete.

[b] The Boolean satisfiability is a part of Cook’s Theorem (1971), which states that every problem in NP is polynomially (in input/circuit family) reducible to CIRCUIT-SAT.

2 Quantum Fourier Transform

Quantum Fourier Transform (QFT) or Fourier Sampling is the quantum analogue of the Discrete Fourier Transform (DFT). Most Quantum Algorithms derive their exponential speed by virtue of QFT. QFT lets us define a state in a new computational basis. From a Physics point of view, the relationship between the two bases is analogous to that between the momentum and position bases of a particle. The mathematical representation of DFT is:

$$\boxed{y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{\frac{2\pi ijk}{N}}}$$

Let us consider an initial state vector:

$$|\psi\rangle = \sum_{j=0}^{N-1} a_j |j\rangle = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_N - 1 \end{pmatrix}$$

The Quantum Fourier Transform is applied on the above state as:

$$F|\psi\rangle = \sum_{j=0}^{N-1} b_k |j\rangle$$

where

$$b_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{\frac{2\pi ijk}{N}}$$

It is clear that Discrete Fourier Transform (DFT) is performed on the amplitudes of the quantum state, through a linear transformation. The F operator can be written in outer product notation as:

$$\boxed{F = \frac{1}{\sqrt{N}} \sum_{j,k=0}^{N-1} e^{\frac{2\pi ijk}{N}} |k\rangle \langle j|}$$

The F operator in matrix form is as follows:

$$QFT_N = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ 1 & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix}$$

where $\omega = e^{\frac{i2\pi}{N}} = cos\frac{2N}{\pi} + \iota sin\frac{2N}{\pi}$

In general we see that the matrix for Quantum Fourier Transform has the form:

$$QFT_N(i,j) = \omega^{ij}$$

Two-Qubit QFT in matrix form:

$$F = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{pmatrix}$$

$$= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -i \\ 1 & -i & -1 & i \end{pmatrix}$$

Note that the QFT matrix is a unitary operator since $FF^\dagger = F^\dagger F = I$.

Complexity:

$$QFT_N = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ 1 & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_N - 1 \end{pmatrix} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_N - 1 \end{pmatrix}$$

To calculate β_0 time required $= O(N)$
To calculate β_1 time required $= O(N)$
$\vdots$
To calculate β_{N-1} time required $= O(N)$

Hence an ordinary algorithm (or a human) would take $O(N^2)$ steps to compute the Fourier Transform of a state. The Fast Fourier Transform (FFT) algorithm (or Cooley-Tuckey algorithm) is a classical algorithm which can compute the DFT of a vector in $O(N \log N)$ steps. The Quantum Fourier Transform (QFT) takes $O(n^2)$ steps at maximum. In our original consideration of n-qubit system, we had $N = 2^n$. Hence we can replace the term n by $\log_2 N$.
Hence the computational complexity of performing a QFT is $O(\log^2 N)$ or $O(2 \log N) = O(\log N)$.
Note that this is an exponential speedup as compared to the classical FFT or FFTW algorithms.

Product Representation of Quantum Fourier Transform:

$$|j\rangle \to \frac{1}{\sqrt{N}} \sum_{k=0}^{k=N-1} e^{\frac{\iota 2\pi kj}{N}} |k\rangle$$

$$\Rightarrow |j_1 \cdots j_n\rangle \to \frac{1}{2^{n/2}} \sum_{k=0}^{k=2^{n/2}-1} e^{\frac{\iota 2\pi j}{2^{n/2}}} |k_1 \cdots k_n\rangle$$

We know, any binary fraction can be written as $\sum_{l=1}^{n} k_l 2^{-l}$

$$\Rightarrow |j_1 \cdots j_n\rangle \to \frac{1}{2^{n/2}} \sum_{k=0}^{k=2^{n/2}-1} e^{\iota 2\pi j \sum_{l=1}^{n} k_l 2^{-l}} |k_1 \cdots k_n\rangle$$

$$= \frac{1}{2^{n/2}} \sum_{k=0}^{k=2^{n/2}-1} \bigotimes_{l=1}^{n} e^{\iota 2\pi j k_l 2^{-l}} |k_l\rangle \qquad (|k_1 \cdots k_n\rangle = |k_1\rangle \otimes |k_2\rangle \otimes \cdots \otimes |k_n\rangle)$$

We can break the 2^n terms in the summation and replace it by n summations over an n-bit binary string.

$$\Rightarrow |j_1 \cdots j_n\rangle \to \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} \bigotimes_{l=1}^{n} e^{\iota 2\pi j k_l 2^{-l}} |k_l\rangle$$

n summation terms can be grouped to one by tensoring it n times.

$$\Rightarrow |j_1 \cdots j_n\rangle \to \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} \left[\sum_{k_l=0}^{1} e^{\iota 2\pi j k_l 2^{-l}} |k_l\rangle \right]$$

$$\Rightarrow |j_1 \cdots j_n\rangle \to \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} \left[|0\rangle + e^{\iota 2\pi j 2^{-l}} |1\rangle \right]$$

We can represent $j2^{-l}$ as a binary fraction. Thus the resulting equation is:

$$\boxed{|j\rangle \to \frac{1}{2^{n/2}} \left[(|0\rangle + e^{\iota 2\pi 0.j_n} |1\rangle) \otimes (|0\rangle + e^{\iota 2\pi 0.j_{n-1} j_n} |1\rangle) \cdots \otimes (|0\rangle + e^{\iota 2\pi 0.j_1 j_2 \cdots j_n} |1\rangle) \right]}$$

3 Walsh-Hadamard Transform

Unlike classical algorithms in which the system is in a specified and deterministic state, quantum algorithms deal with systems which are in a distribution of various states simultaneously with a certain probability of being in each state.
The Walsh-Hadamard transformation, which is closely related to the Quantum Fourier Transformation (QFT) discussed earlier, is an important part of Grover's quantum mechanical algorithm. Using a Walsh-Hadamard transformation, the Diffusion operator (to be discussed later) can be implemented as a product of three unitary transformations. In this project, the Walsh-Hadamard transform has been replaced by the Quantum Fourier Transform (QFT).

$$\boxed{D = WRW = H^{\otimes N} R H^{\otimes N}}$$

Let us define a unitary operator M for a one-qubit system.

$$M = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Let us analyse the action of the M operator on the basis states $|0\rangle$ and $|1\rangle$.

$$M|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = |+\rangle$$

$$M|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = |-\rangle$$

For an n-qubit system, we can have $N = 2^n$ possible states. Hence the state transition matrix M will have dimensions $N \times N$ or $2^n \times 2^n$. Each term in the transformed state will have an amplitude $\frac{1}{\sqrt{N}}$ or $2^{\frac{-n}{2}}$.
On an arbitrary n-bit binary string describing a state, the result of performing an M-transform is that the superposition goes to a n-bit binary string with amplitude $2^{\frac{-n}{2}}$ and sign either $+$ or $-$. The sign is determined by the parity of the bitwise dot product between $\bar{x}$ and $\bar{y}$.

$$\bar{x} \xrightarrow{M} \bar{y}$$

$$parity = (-1)^{\bar{x}.\bar{y}}$$

The formal definition of Walsh-Hadamard Transform [6] matrix W is as follows:

$$\boxed{W_{ij} = 2^{\frac{-n}{2}} (-1)^{\bar{i}.\bar{j}}}$$

4 Selective Rotation

Preferential rotation of the phase of the amplitude in the states of a system can be brought about by a $2^n \times 2^n$ diagonal unitary operator. For an two-qubit system, such an operator will look something like the following:

$$R = \begin{bmatrix} e^{j\phi_1} & 0 & 0 & 0 \\ 0 & e^{j\phi_2} & 0 & 0 \\ 0 & 0 & e^{j\phi_3} & 0 \\ 0 & 0 & 0 & e^{j\phi_3} \end{bmatrix}$$

It is clear that $RR^\dagger = R^\dagger R = I$ since each complex term in the matrix has magnitude 1.

5 Grover's algorithm: Overview

Problem Definition:
In mathematical terms, the problem of unstructured search can be defined by the following quantum oracle.

We are given a database with N records labelled as $0, 1, 2, \cdots, N-1$. We assume $N = 2^n$ for the simple reason of storing each index in n bits. Rather than dealing with list itself, we focus on the *index* of the list. The oracle is thus the following binary function with an n-bit binary string input.

$$f : \{0,1\}^n \rightarrow \{0,1\}$$

$$f(x) = \begin{cases} 1 & \text{for } x = x^* \\ 0 & \text{otherwise} \end{cases}$$

The above oracle f is a blackbox function, which can be queried as many times as required. Each query to the oracle has a computational overhead. The goal is to find the target label x^* with as the minimum number of queries to the oracle.

Example:
There are many problems which can be rephrased as a search problem. One practical example is the random brute force attack on a Data Encryption Standard (DES) encrypted ciphertext [3]. The Key as specified in the DES standard happens to be a 56-bit number.

P (plaintext) = "BlowUpHRI"
C (ciphertext) = "9423180a9fcaa78db8b4d7a3b8f9a9b67e075dc5" (say)
K^* is the 56-bit key used in the DES encryption.

To crack the above ciphertext, we have to perform a brute force with each of the $N = 2^{56}$ keys until we find K^* such that:

$$\boxed{DES(C, K^*) = P}$$

The oracle for the above problem would be:

$$f(K) = \begin{cases} 1 & \text{for } K = K^* \\ 0 & \text{otherwise} \end{cases}$$

Classical algorithms:
$O(N)$ expcected time with linear search.
$O(\frac{N}{2})$ expected time with random brute force.
$O(\log N)$ with binary search for databases with structure (sorted).

Quantum algorithm:
Claim is that quantum algorithm for unstructured search will take time $O(\sqrt{N})$. To search a desired element in an unstructured database, a quantum mechanical algorithm will take at least $\Omega(\sqrt{N})$ steps [5]. Hence the number of steps required by Grover's algorithm is within a constant factor of the fastest possible quantum mechanical algorithm.

Worst case:
There is only x such that $f(x) = 1$

6 Grover's Algorithm: Steps

Phase Inversion:
The initial state of the system is a uniform superposition of all states. The phase inversion step inverts the sign of the amplitude of the needle (x^*).

$$\boxed{\sum_x \alpha_x |x\rangle \xrightarrow{U_{phase}} \sum_{x \neq x^*} \alpha_x |x\rangle - \alpha_{x^*} |x\rangle}$$

The oracle function f can be implemented as an output register of a unitary transformation U_{phase} as:

$$\sum_x \alpha_x |x\rangle |0\rangle \rightarrow \sum_x \alpha_x |x\rangle |0 \oplus f(x)\rangle = \sum_x \alpha_x |x\rangle |f(x)\rangle$$

We can use a small trick to implement phase inversion in the uniform superposition by using the state $|-\rangle$ as the answer bit.

$$\sum_x \alpha_x |x\rangle |-\rangle \rightarrow \sum_x \alpha_x |x\rangle |0 \oplus f(x)\rangle = \sum_x \alpha_x |x\rangle |f(x)\rangle$$

$$\Rightarrow \sum_x \alpha_x |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \xrightarrow{U_{phase}} \sum_x \frac{\alpha_x}{\sqrt{2}} \Big(|x\rangle |0 \oplus f(x)\rangle - |x\rangle |1 \oplus f(x)\rangle \Big)$$

$$\Rightarrow \sum_x \alpha_x |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \xrightarrow{U_{phase}} \sum_x \frac{\alpha_x}{\sqrt{2}} \Big(|x\rangle |f(x)\rangle - |x\rangle \left|\overline{f(x)}\right\rangle \Big)$$

$$\Rightarrow \sum_x \alpha_x |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \xrightarrow{U_{phase}} \sum_x \alpha_x |x\rangle (-1)^{f(x)} \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)$$

It is evident that the oracle qubit $|-\rangle$ is always unchanged and hence can be omitted from the above equation.

$$\Rightarrow \boxed{\sum_x \alpha_x |x\rangle \xrightarrow{U_{phase}} \sum_x \alpha_x |x\rangle (-1)^{f(x)}}$$

The above operation can be represented by a unitary operator as:

$$\boxed{I_t = I - 2|t\rangle\langle t|}$$

Reflection about mean:
Though phase inversion logically separates the search element from the rest of the haystack, it doesn't help in increasing the probability of the outcome corresponding to the "needle" when a measurement is performed. To solve this problem, we do a reflection about mean.

Before defining an operator to perform the above transformation, let us consider an operator S which does a reflection about $|0^n\rangle$. The unitary transformation describing this reflection can be represented as:

$$\boxed{S = 2|0^n\rangle\langle 0^n| - I}$$

$$S|x\rangle = (2|0^n\rangle\langle 0^n| - I)|x\rangle$$

Assuming a single qubit system, we can construct the S operator as:

$$\Rightarrow S|x\rangle = \left(2\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 1 & 0 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right)|x\rangle$$

$$\Rightarrow S|x\rangle = \left(2\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right)|x\rangle$$

$$\Rightarrow S|x\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}|x\rangle$$

$$\Rightarrow \boxed{S|x\rangle = \sigma_z|x\rangle}$$

Another fancy way of writing the unitary operator $|S\rangle$ for reflection about $|0^n\rangle$ is:

$$\boxed{|x\rangle \rightarrow -(-1)^{\delta_{x,0}}|x\rangle}$$

where, $\delta_{x,0}$ is the Kronecker Delta function for orthonormality condition.

$$|0\rangle \rightarrow -(-1)^{\delta_{0,0}}|0\rangle = |0\rangle$$

$$|1\rangle \rightarrow -(-1)^{\delta_{1,0}}|0\rangle = -|1\rangle$$

If the initial superposition is of the form $\sum_x \alpha_x|x\rangle$, the mean of the superposition is given by:

$$\boxed{\mu = \frac{\sum_x^{N-1} \alpha_x}{N}}$$

In general, the reflection about mean μ can be carried out by first mapping $|\psi\rangle$ to $|0^n\rangle$ by applying $H^{\otimes n}$ or the Quantum Fourier Transform (QFT), then applying the S operator defined above, and then mapping $|0^n\rangle$ back to $|\psi\rangle$ by applying $H^{\otimes n}$ or the Quantum Fourier Transform (QFT) again.

$$\boxed{D = H^{\otimes n}SH^{\otimes n}}$$

Details on the Diffusion operator is given in the next section.

7 Grover's Algorithm: Implementation

Algorithm:
1. Start the state in $|\psi\rangle \sum_x \frac{1}{\sqrt{N}}|x\rangle$.
2. Invert the phase of the search element by applying the oracle f.
3. Apply the Quantum Fourier Transform ($H^{\otimes n}$).
4. Apply conditional phase shift about $|0^n\rangle$.
5. Apply inverse Quantum Fourier Transform ($H^{\otimes n}$).
6. Repeat steps 2 to 5 for $O(\sqrt{N})$ steps.

The steps 3-5 can be combined into a single unitary transformation I_s such that the net Grover's operator becomes $G = I_s I_t$:

$$D = H^{\otimes n}\big[2|0^n\rangle\langle 0^n| - I\big]H^{\otimes n}$$

$$\Rightarrow \boxed{I_s : D = 2|\psi\rangle\langle\psi| - I}$$

The above operator is also known as *Diffusion operator*, which can be represented in matrix form as:

$$D = H^{\otimes n}\left(2\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix} - I\right)|H^{\otimes n}$$

$$D = H^{\otimes n}\left(\begin{bmatrix} 2 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix} - I\right)H^{\otimes n}$$

$$D = H^{\otimes n}\begin{bmatrix} 2 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix} H^{\otimes n} - H^{\otimes n} I H^{\otimes n}$$

$$D = \frac{1}{\sqrt{N}}\begin{pmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \cdots & \omega^{2(N-1)} \\ 1 & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \omega^{3(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix}\begin{bmatrix} 2 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix} H^{\otimes n} - I$$

$$D = \begin{bmatrix} \frac{2}{\sqrt{N}} & 0 & 0 & \cdots & 0 \\ \frac{2}{\sqrt{N}} & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{2}{\sqrt{N}} & 0 & 0 & \cdots & 0 \end{bmatrix} H^{\otimes n} - I$$

$$D = \begin{bmatrix} \frac{2}{\sqrt{N}} & 0 & 0 & \cdots & 0 \\ \frac{2}{\sqrt{N}} & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{2}{\sqrt{N}} & 0 & 0 & \cdots & 0 \end{bmatrix}\frac{1}{\sqrt{N}}\begin{pmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \cdots & \omega^{2(N-1)} \\ 1 & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \omega^{3(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix} - I$$

$$D = \begin{bmatrix} \frac{2}{N} & \frac{2}{N} & \frac{2}{N} & \cdots & \frac{2}{N} \\ \frac{2}{N} & \frac{2}{N} & \frac{2}{N} & \cdots & \frac{2}{N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{2}{N} & \frac{2}{N} & \frac{2}{N} & \cdots & \frac{2}{N} \end{bmatrix} - I$$

$$I_s : D = \begin{bmatrix} (\frac{2}{N}-1) & \frac{2}{N} & \frac{2}{N} & \cdots & \frac{2}{N} \\ \frac{2}{N} & (\frac{2}{N}-1) & \frac{2}{N} & \cdots & \frac{2}{N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{2}{N} & \frac{2}{N} & \frac{2}{N} & \cdots & (\frac{2}{N}-1) \end{bmatrix}$$

The D operator is a unitary transformation and hence can act on a Ket vector to give another Ket vector.

$$D \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_N - 1 \end{pmatrix} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_N - 1 \end{pmatrix}$$

It is easy to check that $\beta_i = \frac{-2}{N}\sum_j \alpha_j + \alpha_i = -2\mu + \alpha_i$. Thus the Diffusion operator D can flip the amplitudes of the superposition about the mean μ.

$$\alpha_x \xrightarrow{U_{mean}} 2\mu - \alpha_x = \mu + (\mu - \alpha_x)$$

$$\Rightarrow \boxed{\sum_x \alpha_x |x\rangle \xrightarrow{U_{mean}} \sum_x (2\mu - \alpha_x)|x\rangle}$$

The projection matrix $P = |\psi\rangle\langle\psi|$ in the Diffusion operator has the following properties:
1. $P^2 = P$, where $P_{ij} = \frac{1}{N}$.
2. P acting on any vector $\bar{v}$ gives a vector each of whose components is equal to average of all components.

Using the first property of the projection matrix, it can be shown that the Diffusion operator is a matrix which conserves probabilities [7].

$$D = 2P - I$$
$$D^2 = (2P - I)(2P - I)$$
$$= 4P^2 - 4PI + I^2$$
$$= 4PI - 4PI + I$$
$$\Rightarrow DD = I$$
$$\Rightarrow D = D^\dagger$$
$$\boxed{DD^\dagger = D^\dagger D = I}$$

This confirms the Diffusion operator to be unitary as was shown earlier.

Alternative algorithm (using Diffusion operator):
1. Start the state in $|\psi\rangle \sum_x \frac{1}{\sqrt{N}}|x\rangle$.
2. Invert the phase of the search element by applying the oracle f.
3. Invert about mean using the Diffusion operator D.
4. Repeat steps 2 and 3 for $O(\sqrt{N})$ steps.

8 Grover's Algorithm: Geometric interpretation

Let us consider a database of N items having M solutions. Thus, the initial superposition can be written as:

$$|\psi\rangle = \sum_x \alpha_x |x\rangle = a|\alpha\rangle + b|\beta\rangle$$

where, $|\alpha\rangle = \frac{1}{\sqrt{N-M}} \sum_{\{x|f(x)\neq 1\}} |x\rangle$ and $|\beta\rangle = \frac{1}{\sqrt{M}} \sum_{\{x|f(x)=1\}} |x\rangle$.

Thus $|\alpha\rangle$ is a superposition of the states which do not satisfy the oracle, and $|\beta\rangle$ is a superposition of states which satisfy the oracle. The state $|\psi\rangle$ can be re-written in terms of $|\alpha\rangle$ and $|\beta\rangle$ in normalized form as:

$$\boxed{|\psi\rangle = \sqrt{\frac{N-M}{N}}|\alpha\rangle + \sqrt{\frac{M}{N}}|\beta\rangle}$$

So the initial state is in a space spanned by $|\alpha\rangle$ and $|\beta\rangle$. Since $a^2 + b^2 = 1$, we can express the amplitudes of $|\psi\rangle$ as sinusoids.

$$\boxed{|\psi\rangle = \cos\frac{\theta}{2}|\alpha\rangle + \sin\frac{\theta}{2}|\beta\rangle}$$

Phase inversion:
The phase inversion operation on a state vector can be thought of as a rotation of the state by an angle -θ clockwise (or θ anticlockwise).

$$a|\alpha\rangle + b|\beta\rangle \xrightarrow{oracle} a|\alpha\rangle - b|\beta\rangle$$

$$\Rightarrow |\psi\rangle \xrightarrow{oracle} \cos\frac{\theta}{2}|\alpha\rangle - \sin\frac{\theta}{2}|\beta\rangle$$

$$\boxed{|\psi\rangle \xrightarrow{oracle} \cos\frac{-\theta}{2}|\alpha\rangle + \sin\frac{-\theta}{2}|\beta\rangle}$$

Reflection about mean:
The Grover operator G reflects the $O|\psi\rangle$ state about the mean $|\psi\rangle$. This operation also includes the unitary transformation which performs phase inversion of the search element.

$$\Rightarrow G|\psi\rangle = \cos\frac{3\theta}{2}|\alpha\rangle + \sin\frac{3\theta}{2}|\beta\rangle$$

In general, for k iterations,

$$\boxed{G^k|\psi\rangle = \cos\frac{(2k+1)\theta}{2}|\alpha\rangle + \sin\frac{(2k+1)\theta}{2}|\beta\rangle}$$

Grover operator $G = 2|\psi\rangle\langle\psi| - I$ performs reflection in the plane defined by $|\alpha\rangle$ and $|\beta\rangle$ about the vector $|\psi\rangle$. So the $G^k|\psi\rangle$ remains in the space spanned by $|\alpha\rangle$ and $|\beta\rangle$ for all k. Repeated rotation of the state vector using the Grover operator brings it close to $|\beta\rangle$. A subsequent measurement in the computational basis produces one of the outcomes in the superposition $|\beta\rangle$ with high probability, thus finding a solution in the search stack.

9 Grover's Algorithm: Complexity

Let the rotation brought about by the Grover operator be repeated for k number of times. We are interested in finding the minimum value of this number k. The effect of Grover's operator G on the initial state is given below [9].

$$\boxed{G^k|\psi\rangle = \cos\frac{(2k+1)\theta}{2}|\alpha\rangle + \sin\frac{(2k+1)\theta}{2}|\beta\rangle}$$

Ideally we want to get as close as possible to $|\beta\rangle$, with the smallest possible value of k. This means that we want the value of $\sin\frac{(2k+1)\theta}{2}$ to be as close as possible to 1. It follows that the ideal value of k would satisfy the following condition:

$$(2k+1)\frac{\theta}{2} = \frac{\pi}{2}$$

$$(2k+1)\phi = \frac{\pi}{2}$$

$$\Rightarrow \boxed{k = \frac{\pi}{4\phi} - \frac{1}{2}}$$

Since it is logical for k to be an integer, we can round it to the nearest integer using the floor function.

$$k = round(\frac{\pi}{4\phi} - \frac{1}{2}) = \lfloor\frac{\pi}{4\phi}\rfloor$$

In worst case scenario there is only one x which satisfies the oracle defined earlier, i.e., $M = 1$.

$$\sin\phi = \langle\beta|\psi\rangle = \sqrt{\frac{M}{N}} = \sqrt{\frac{1}{N}}$$

A digital haystack can potentially be very huge (large value of N). Thus we can consider the following approximation since the value of $\sqrt{\frac{1}{N}}$ is very small [3].

$$\sin\phi = \phi = \sqrt{\frac{1}{N}}$$

$$\Rightarrow \boxed{k = \left\lfloor\frac{\pi\sqrt{N}}{4}\right\rfloor}$$

Thus we need $O(\sqrt{N})$ Grover iterations for the algorithm to complete. The algorithm succeeds with probability $O(1)$ [9].

The paper [5] suggests that it is impossible to search an element in a digital database in fewer than $\Omega\sqrt{N}$ steps. A tight bound of the quantum search algorithm has been given in the paper [8] showing that Grover's search algorithm is close to the fastest possible quantum mechanical algorithm.

10 Mathematical Analysis of Grover's iterations

Let the state vector be defined as follows:

$$|\psi\rangle = \begin{pmatrix} l \\ l \\ \vdots \\ k \\ \vdots \\ l \end{pmatrix}$$

where k and l are real numbers and number of states having amplitude l is $N-1$.
From the definition of the Diffusion operator, we can apply the rules of matrix multiplication to get the following equations [1]:

$$D \begin{pmatrix} l \\ l \\ \vdots \\ k \\ \vdots \\ l \end{pmatrix} = \begin{pmatrix} l_1 \\ l_1 \\ \vdots \\ k_1 \\ \vdots \\ l_1 \end{pmatrix}$$

$$\begin{bmatrix} \left(\frac{2}{N}-1\right) & \frac{2}{N} & \frac{2}{N} & \cdots & \frac{2}{N} \\ \frac{2}{N} & \left(\frac{2}{N}-1\right) & \frac{2}{N} & \cdots & \frac{2}{N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{2}{N} & \frac{2}{N} & \frac{2}{N} & \cdots & \left(\frac{2}{N}-1\right) \end{bmatrix} \begin{pmatrix} l \\ l \\ \vdots \\ k \\ \vdots \\ l \end{pmatrix} = \begin{pmatrix} l_1 \\ l_1 \\ \vdots \\ k_1 \\ \vdots \\ l_1 \end{pmatrix}$$

$$\Rightarrow \boxed{k_1 = \left(\frac{2}{N}-1\right)k + \left(\frac{2}{N}\right)(N-1)\,l}$$

and

$$\boxed{l_1 = \left(\frac{2}{N}-1\right)l + \left(\frac{2}{N}\right)k + \left(\frac{2}{N}\right)(N-2)\,l}$$

The second equation can also be reduced to:

$$l_1 = \left(\frac{2}{N}\right)k + \left(\frac{N-2}{N}\right)l$$

Corollary 1 [1]
Let k be negative, and l be positive. Assume $N > 2$. It follows that $\left(\frac{2}{N}-1\right)$ is negative. Also for our assumption of $N > 2$, $\left(\frac{2}{N}\right)(N-1)$ is positive. Thus the expression for k_1 becomes (negative $\times$ negative) + (positive $\times$ positive), which is positive.

Corollary 2 [1]
Squaring both sides of the equation $k_1 = \left(\frac{2}{N}-1\right)k + \left(\frac{2}{N}\right)(N-1)\,l$, we get the following:

$$k_1 = \frac{(N-2)^2}{N^2}k^2 + 4\frac{(N-1)^2}{N^2}l^2 - 4\frac{(N-2)(N-1)}{N^2}kl$$

Squaring both sides of equation $l_1 = \left(\frac{2}{N}\right)k + \left(\frac{N-2}{N}\right)l$ and multiplying $(N-1)$, we get:

$$(N-1)l_1^2 = 4\frac{(N-1)^2}{N^2}k^2 + \frac{(N-2)^2}{N^2}(N-1)l^2 + 4\frac{(N-2)(N-1)}{N^2}kl$$

Adding the two equations, we get:

$$k_1^2 + (N-1)l_1^2 = k^2 + (N-1)l^2$$

The above corollary is a very important result which follows from the fact that the Diffusion operator is a unitary transformation, the sum of the absolute squares of the components is same before and after the transformation.

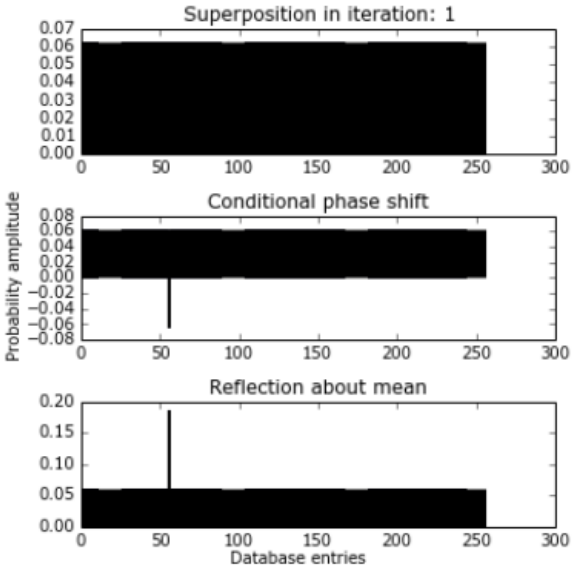
11 Appendix: Helper functions

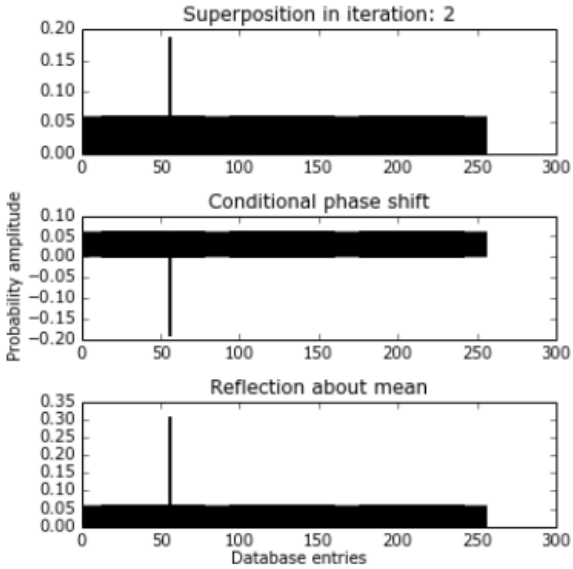
```
# Helper functions implemented in Python

def U_phase(psi, e):
    """
    Unitary transformation to invert the phase of the search element

    Parameters                | Returns
    --------------------------|----------------------------------
    psi : ndarray             | psi : ndarray
          Quantum state       |       State after phase inversion
    e   : int                 |
          Search element      |
    """
    if psi.shape[0] >= e:
        psi[e] *= -1
    return psi

def QFT(psi, N):
    """
    Quantum Fourier Transform operator in N dimensions

    Parameters                | Returns
    --------------------------|---------------------------
    psi : ndarray             | QFT : ndarray
          Quantum state       |       QFT unitary operator
    N   : int                 |
          Number of dimensions|

    Internals
    ---------
    Makes use of the DFT method of NumPy. QFT is a unitary operator
    so the QFT matrix is its own inverse.
    """
    return np.fft.fftn(psi)/N**0.5

def Diffusion(N):
    """
    Diffusion operator D for Grover's iterations

    Parameters                | Returns
    --------------------------|------------------
    N   : int                 | D   : ndarray
          Number of dimensions|       Unitary operator D
    """
    return np.ones((N, N)) * 2/N - np.identity(N)

def __iter(N):
    """
    Number of iterations to run the Grover's operators

    Parameters                | Returns
    --------------------------|------------------
    k  : int                  | N  :  int
         Number of dimensions |       Number of Grover iterations
    """
    return int(np.floor(np.pi * (N**0.5) /4))
```

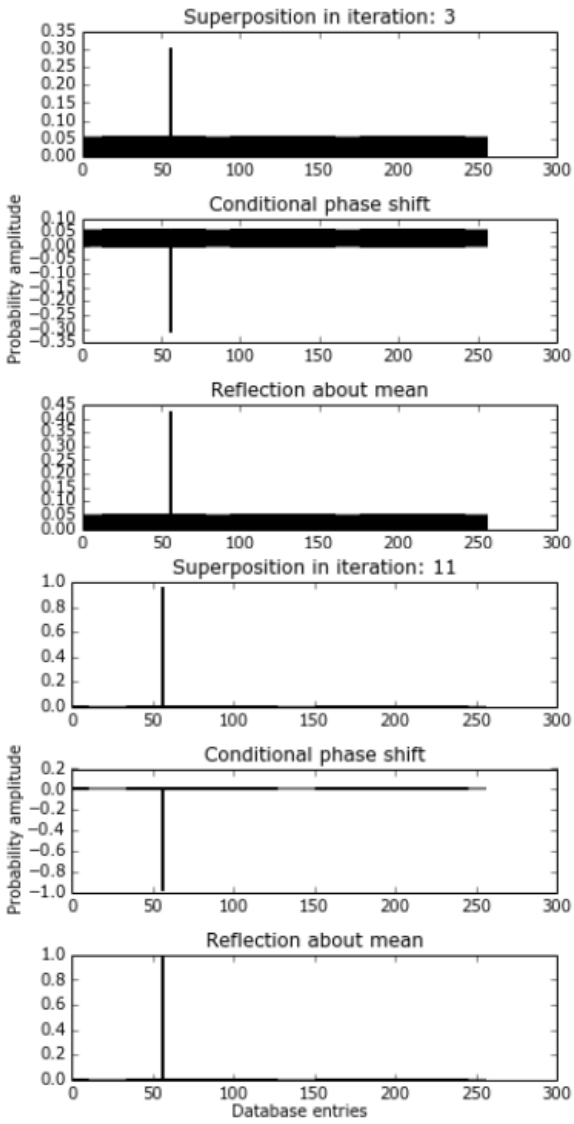
12 Appendix: Grover's algorithm examples

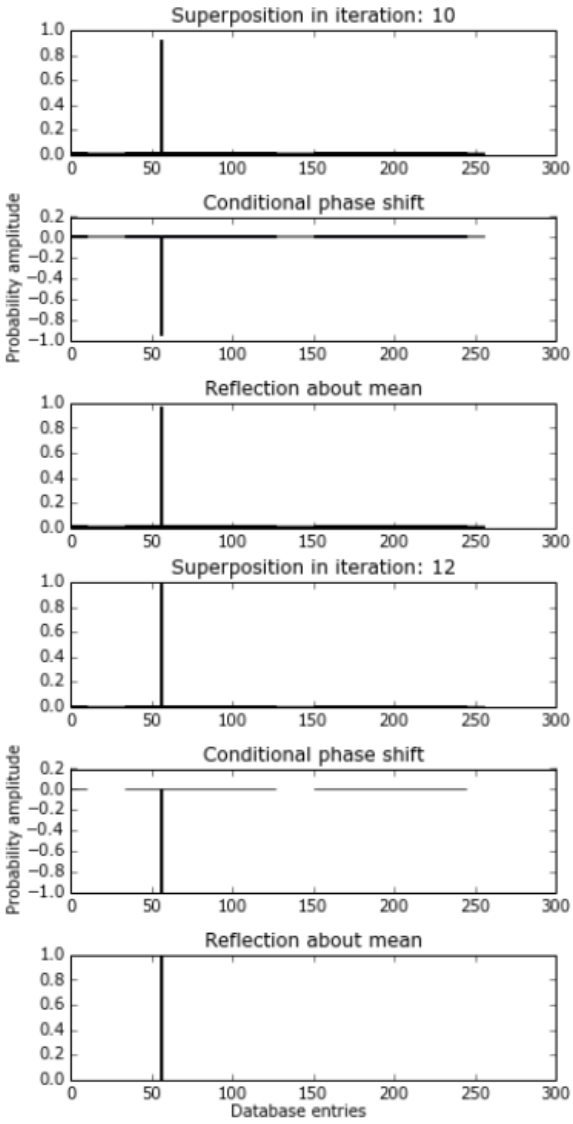
```
# Grover's iteration for one qubit system
N=4
x=2

psi = np.ones((N, 1))/N**0.5
psi = U_phase(psi, x)
psi = QFT(psi, N) # QFT on state psi

si = np.zeros((N, 1))
si[0][0] = 1 # The |0> state
si_dag = np.conjugate(si.T) # The <0| state
S = 2 * np.kron(si, si_dag) - np.identity(N) # S = (2|0><0|-I)

psi = np.dot(S, psi) # Conditional phase shift operation
psi = QFT(psi, N) # Inverse QFT on state psi

psi

>>> array([[ 0.+0.j],
       [ 0.+0.j],
       [ 1.+0.j],
       [ 0.+0.j]])


# Grover's iterations for 8 qubit system
N = 256
x = 55

D = Diffusion(N)
k = __iter(N) # Estimate least number of iterations required
probability_amplitude_list = [1/(N**0.5)]
psi = np.ones((N, 1))/N**0.5

for c in xrange(k):
    fig = plt.figure(figsize=(5, 5))
    ax1 = fig.add_subplot(311)
    plt.bar(np.arange(N), psi.T[0])
    ax1.set_title("Superposition in iteration: %s"%(c+1))

    psi = U_phase(psi, x) # Conditional phase shift
    ax2 = fig.add_subplot(312)
    ax2.set_title("Conditional phase shift")
    plt.bar(np.arange(N), psi.T[0])

    psi = np.dot(D, psi) # Reflection about mean
    ax3 = fig.add_subplot(313)
    ax3.set_title("Reflection about mean")
    plt.bar(np.arange(N), psi.T[0])

    fig.text(0.5, 0.01, 'Database entries', ha='center', va='center')
    fig.text(0.01, 0.5, 'Probability amplitude',
             ha='center', va='center', rotation='vertical')
    fig.tight_layout()
    probability_amplitude_list.append(psi[x][0])
```

13 Grover's Algorithm: Plots



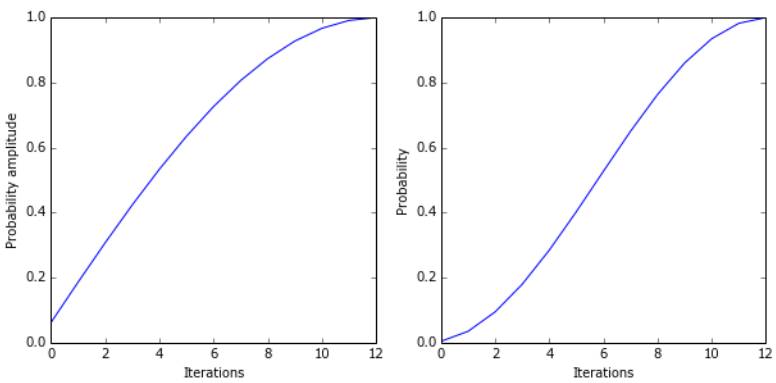






14 Appendix: Amplification in Grover's algorithm

```
probability_list = [i ** 2 for i in probability_amplitude_list]
fig = plt.figure(figsize=(10, 10))
ax1 = fig.add_subplot(221)
ax1.set_xlabel("Iterations")
ax1.set_ylabel("Probability amplitude")
plt.plot(np.arange(k+1), probability_amplitude_list)
ax2 = fig.add_subplot(222)
ax2.set_xlabel("Iterations")
ax2.set_ylabel("Probability")
plt.plot(np.arange(k+1), probability_list)
print '''Probability amplitudes\n----------------------'''
pprint.pprint(probability_amplitude_list)
print '''Probabilities\n----------------------'''
pprint.pprint(probability_list)

>>> Probability amplitudes
----------------------
[0.0625,
 0.1865234375,
 0.3076324462890625,
 0.4239346981048584,
 0.53361297026276588,
 0.63495353976031765,
 0.72637296019911446,
 0.8064428031348001,
 0.87391197727150449,
 0.9277262767633413,
 0.96704485318074529,
 0.99125335376719736,
 0.99997352070104339]
Probabilities
----------------------
[0.00390625,
 0.034790992736816406,
 0.094637722009792924,
 0.17972062825725743,
 0.28474280203265145,
 0.40316599765415728,
 0.52761767730842435,
 0.65034999472791399,
 0.76372214401859062,
 0.86067604459717173,
 0.93517574806336923,
 0.98258321135471649,
 0.99994704210324004]
```



15 References

Papers

[1] Lov K. Grover, *"A fast quantum mechanical algorithm for database search"*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212, 1996 [arXiv.org:quant-ph/9605043]

[2] Lov K. Grover, *"Quantum Mechanics helps in searching for a needle in a haystack"*, Physical Review Letters, Volume 79, Number 2, pp. 325-328, 1997 [arXiv.org:quant-ph/9706033]

[3] Samuel J. Lomonaco Jr., *"Grover's Quantum Search Algorithm"*, Proceedings of Symposia in Applied Mathematics

[4] Lov K. Grover, *"Quantum computer can search arbitrarily large databases by a single query"*, Physical Review Letters, pp 4709-4712, 1997.

[5] C. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, *"Strengths and Weaknesses of Quantum Computing"*, SIAM Journal of Computing, 26, 1510, 1997 [arXiv.org:quant-ph/9701001]

[6] D. Deutsch and R. Jozsa, *"Rapid solution of problems by quantum computation"*, in Proceedings Royal Society of London, A400, pp. 73-90, 1992.

[7] E. Bernstein and U. Vazirani, *"Quantum Complexity Theory"*, Proceedings 25th ACM Symposium on Theory of Computing, pp. 11-20, 1993.

[8] M. Boyer, G. Brassard, P. Hoyer, A. Tapp, *"Tight bounds on quantum searching"*, Proceedings PhysComp 1996. [lanl eprint quant-ph/9605034]

Books

[9] M. A. Neilsen and I. L. Chuang, *"Quantum Computation and Quantum Information"*, Cambridge University Press, 2000.

Course notes

[10] Dieter van Melkebeek, *"Quantum Search"*, CS880: Quantum Information Processing, University of Wisconsin-Madison, Fall 2010

[11] Umesh Vazirani, *"Quantum Search + Quantum Zeno effect"*, CS191: Quantum Information, University of California-Berkeley, Spring 2012